

Lecture 5: Media Device Software in a Nutshell

Lecture 5 marks the beginning of the second part of the course on software for digital TV and multimedia. In the first part of the course, the main focus was on hardware terminology, commands, abstract processing blocks and signal-processing phases. We looked at what the device needs to do, which blocks exist, how the signal path is organized and how the CPU orchestrates the complete process.

The focus now shifts more explicitly into the software space.

The previous model still stands. We are not replacing the production-distribution-consumption view, nor the reception-processing-presentation model on the consumer side. Instead, we are asking a more practical question: when a concrete software stack exists, how are those commands and data exchanges actually implemented? Which libraries are used? Which software layers exist? Which execution environments are involved? Where does the application end, where does middleware begin, and where do hardware abstractions appear?

This lecture answers those questions by introducing the host software stack used in digital TV and multimedia devices.

Hardware Blocks vs Host Software

We remain positioned inside the same model used in the previous lectures: production, distribution and consumption. More specifically, we are still focused on the consumer side: the TV, set-top box, or application running on a media-capable device.

The earlier device diagram decomposed the consumer device into reception, processing and presentation stages (Figure 1). At the input side, there are front-end blocks. Then comes the demultiplexer. If the content is protected, conditional access or descrambling appears as an extension of demultiplexing. The central processor performs command-and-control tasks. Separate audio and video decoders handle media decoding. Additional AI accelerators or DSP units may perform specialized processing. Finally, the GPU handles composition, blending and 2D or 3D acceleration before the final image is sent to the display.

Most of these blocks, except for components such as the tuner, display, speakers, power supply and external memory banks, are commonly integrated into a single chip. This chip is the System on Chip, or SoC.

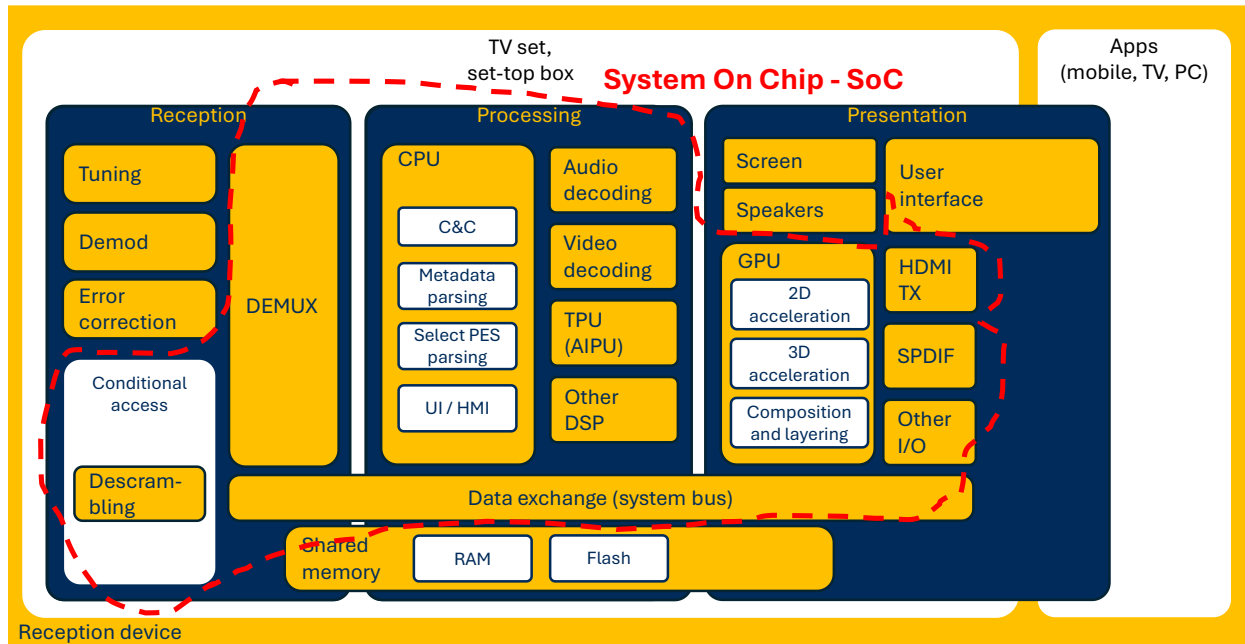


Figure 1. Device architecture with the consumer – recap

The important question now is: where is the software?

Software exists throughout the system. A video decoder may contain internal firmware. A DSP may run dedicated routines. An AI accelerator may have its own execution model. However, when we talk about the complex software that application and embedded engineers usually develop, we mostly talk about software running on the central processor.

In previous diagrams, the CPU area contained abstract blocks such as command and control, metadata parsing and other orchestration functions. These were shown as high-level responsibilities. Now we identify the concrete software stack behind them. Collectively, this CPU-side software is commonly called host software.

Host software is the main software running on the CPU. It controls the system, parses metadata, communicates with drivers, coordinates playback and manages the user-facing behavior of the device.

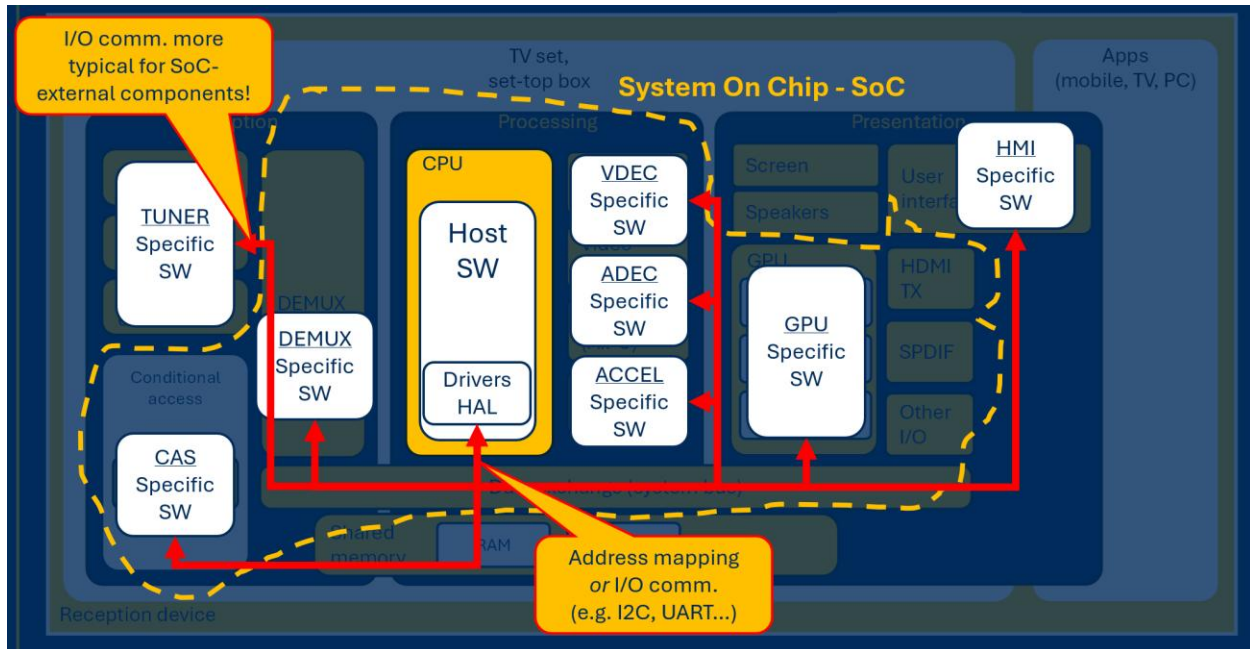


Figure 2. Host SW in relationship to other SW blocks within the device

Why Host Software Matters

In most consumer electronics projects, engineers work at the level of use cases. Consumer electronics always has a user-facing use case and a user interface. Without the user, there is no consumer electronics product.

This is why, in the majority of typical media-device projects, development work happens on the CPU side, inside host software. Engineers usually do not program the internal firmware of the video decoder or the low-level logic of the demux. That type of work may happen inside a SoC company, where one team may focus only on a specialized block, such as a decoder or accelerator.

For system and application engineers building complete media products, the specialized blocks are usually treated as hardware provided by the SoC vendor. The practical development focus is the host software that controls and integrates them.

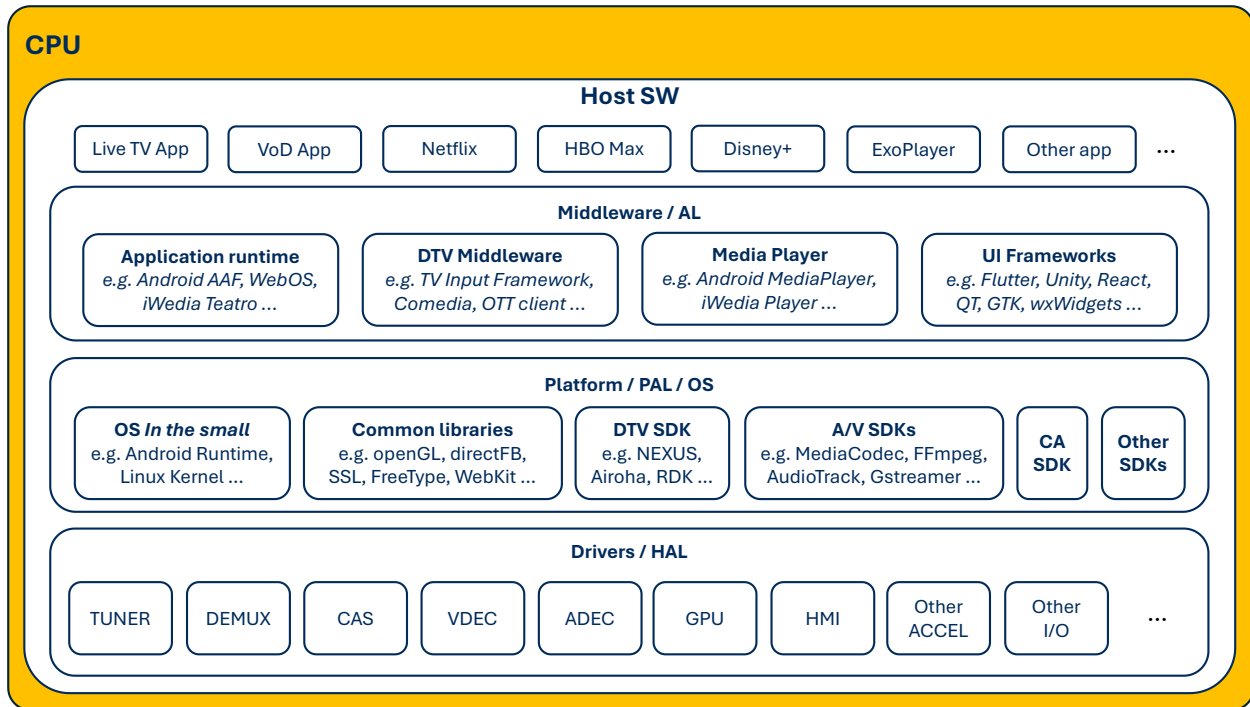


Figure 3. Host software - block diagram of the complete stack

Drivers and Hardware Abstraction Layer

Even when engineers do not program hardware blocks directly, host software still needs to see and control them. This happens through drivers and the Hardware Abstraction Layer, usually abbreviated as HAL.

All individual hardware devices and processing blocks are exposed toward host software through drivers or HAL APIs. In this course, these layers are mostly treated as given. When software needs to communicate with a peripheral component or hardware block, it uses the HAL API or driver abstraction provided by the platform.

Under the hood, drivers use address maps and hardware registers. For most blocks inside the SoC, the driver reads from and writes to memory-mapped registers. It also reacts to interrupts and implements the low-level timing and control behavior needed to interact with the hardware.

In simple terms, driver code interprets the hardware specification and translates it into C functions that perform register reads, register writes and interrupt handling.

If a block is more complex or external to the SoC, the driver may communicate through interfaces such as SPI or I2C. This complexity remains hidden behind the driver or HAL layer.

Two common cases are important. If the hardware block is inside the SoC, it is usually accessed through memory-mapped registers. The CPU reads and writes specific addresses, and those addresses correspond to registers of the hardware block. If the hardware block is outside the SoC, there is usually no direct memory mapping. Communication is then performed through an I/O interface, very often I2C.

This sometimes causes confusion. An external tuner may still have registers, but the CPU does not access them through direct memory mapping. Instead, the CPU talks to the SoC's I2C peripheral, and that peripheral communicates with the tuner's internal registers. So the software still "writes to tuner registers," but indirectly, through I2C.

This is the basic software coupling between host software and the hardware blocks around it.

HAL APIs: What Can the Hardware Do?

Once hardware blocks are represented as drivers or software modules, the naming of those modules usually corresponds directly to the hardware blocks. If there is a video decoder in hardware, there is usually a video decoder driver. If there is a tuner, there is a tuner driver. If there is a demux, there is a demux driver.

At this level, there is usually not too much software philosophy. The HAL layer is not primarily about elegant application-level abstractions or complex design patterns. It exposes what the hardware can actually do.

A useful way to understand the HAL API is to ask a simple question: what can the hardware do?

If the answer is written in normal language, it becomes a description. If the answer is written as function names, it becomes an API.

For example, what can a tuner do? It can tune to a frequency. Therefore, the API may include a function such as `tune`, `setFrequency` or `setParameter`. The tuner can also be initialized, so there is likely an initialization function. It may report lock status, signal strength or other reception parameters, which again become API functions.

The same reasoning applies to other blocks. A demux can filter PIDs. A decoder can be configured and started. A GPU can receive composition commands. The HAL API is the software-shaped answer to the question of what the hardware can do.

Driver Models and Linux Abstractions

The HAL layer can be implemented in different ways. In smaller software stacks, there may be no full operating system. The HAL may simply be a folder of C code built together with the firmware as a monolithic program. In that case, functions may have explicit and verbose names, and upper layers call them directly.

In more complex systems, especially Linux-based ones, hardware support is structured through kernel drivers. Kernel drivers follow standardized patterns and typically expose functions such as `open`, `close`, `ioctl`, `read` and `write`.

This standardization is important because it allows drivers to be dynamically instantiated, managed and connected to the rest of the system. Drivers can be loaded, removed or packaged as modules without rebuilding the entire software stack.

The `ioctl` call is especially flexible. It allows software to send generic control commands to a driver and receive corresponding results. The `read` and `write` calls are more suitable for block communication, especially where larger amounts of data must be delivered or received. This is where the producer-consumer design pattern naturally appears: one side produces data, another consumes it through buffers or stream-like communication.

Even in Linux, the driver code inside the kernel can be understood as a form of HAL. It adapts hardware-specific behavior to standardized operating-system mechanisms.

Platform, PAL and Operating System

The next layer above HAL is the platform, also called the Platform Abstraction Layer, or PAL. The purpose of a platform is to standardize the environment in which software is developed.

People often use terms such as operating system or operating environment for this layer. However, the platform is more than the operating system alone.

Inside the platform, there is what can be called the operating system in the small. This refers to the classical operating-system role: managing the processor, threads, processes, memory allocation, files, file systems and communication mechanisms.

The term operating system can be confusing because ordinary users and engineers often mean different things by it. A user may think of Windows, Android or a complete distribution full of applications and tools. In the engineering sense, the operating system is the lower-level mechanism that manages processes, scheduling, memory, files and communication.

In media devices, two major platform flavors are common. One is the Android flavor. The other is the Linux plus web flavor.

Android is now widely present across phones, TVs and other consumer devices. The Linux flavor is usually paired with a browser-based operating environment. In media devices, this means that the platform is often either an Android environment or Linux with a web-based application environment.

Android and Linux-plus-Web Platforms

In the Linux-plus-web direction, names such as Tizen, webOS and other web-like operating environments appear. In essence, many such systems can be understood as a full-screen

browser without the usual browser chrome, displaying HTML, CSS and JavaScript across the entire screen. This web application then becomes the device application.

Of course, such browsers are extended with plugins, bridges or native services, so that the application can communicate with platform services, middleware and drivers. Underneath, Linux is often the base.

In theory, Android runtime could also be combined with a browser-based application model. In practice, this is usually less logical because Android already provides a complete application framework. If Android is available, it is more natural to use its application model.

On Linux, the situation is different. Other UI frameworks such as Qt or GTK may appear in some industries, especially automotive infotainment, but in TV and media devices the web-based model has become dominant.

The reason is audience reach. Consumer electronics requires applications to work for as many users and devices as possible. The web is the most widespread, widely used and widely debugged application technology. Historically, embedded browsers were difficult and unstable, often suffering from memory leaks. Early embedded browser systems sometimes even required scheduled nightly resets to keep the device stable.

Over time, embedded browser implementations improved. Proprietary embedded browsers became a serious business. Companies could take open-source browser technology, stabilize it, optimize it for embedded devices and sell licenses and support. This model exists because simply taking a generic browser engine such as WebKit may not be sufficient for all device classes. A basic set-top box may tolerate more limitations, while more demanding systems such as automotive infotainment require stronger guarantees and deeper integration.

Common Libraries

The platform also includes common libraries. These are platform libraries and abstractions that solve general classes of problems. They are not specific only to media and appear across many industries.

Communication libraries are one example. SSL is one such component, and the entire TCP/IP stack can also be thought of as a common platform capability. These are widely used infrastructure elements.

Graphics, acceleration and computation libraries are another important group. Examples include OpenGL, DirectFB, OpenCV, OpenCL and OpenVX. These libraries allow software to use hardware more elegantly and to perform accelerated operations.

OpenGL is classically associated with 3D acceleration. Underneath it, there is a GPU. However, every GPU has its own low-level driver details. OpenGL provides a standardized

API above different GPU implementations and hides much of the platform-specific complexity.

This does not mean OpenGL is trivially portable in practice. On an embedded platform, it is usually not enough to build a generic OpenGL implementation from source. The correct approach is normally to link against the OpenGL implementation provided with the BSP, because the SoC maker has optimized it for the specific chip and GPU. A generic software implementation may work but may not use hardware acceleration effectively, which defeats the purpose.

DirectFB is associated with 2D graphics. Fonts, WebKit and similar components are also examples of common libraries commonly found in media device platforms.

Platform APIs: What Does the Programmer Want?

At the HAL level, the guiding question is: what can the hardware do?

At the platform level, the question changes: what does the programmer want?

The user of the platform is not the final consumer. The user of the platform is the developer building applications and services on top of it.

When the answer to “what does the programmer want?” is expressed as functions, it becomes the platform API.

Programmer expectations grow over time. Developers want more convenience, more abstraction and more ready-made functionality. This is why platforms become thicker and thicker. In the past, SoC companies could provide lower layers and still sell chips. Today, they often need to provide a large part of the platform stack as well.

A file is a simple example of a platform abstraction. A file does not exist as a physical object in hardware. Hardware contains memory cells, blocks, sectors and addresses, but not files. The file is a software abstraction built on top of storage.

The same applies to threads and processes. They are not physical objects; they are operating-system abstractions. Even a memory buffer is a programmer abstraction representing allocated memory, which may be virtually continuous even when physically mapped across different locations.

The programmer wants to create a file, delete a file, allocate memory, create a process, wait for an event or draw something on screen. With OpenGL, the wish may be to draw a cube. With DirectFB, it may be to draw a rectangle or write text at an X-Y location. Coordinates, windows, layers and pixels are all abstractions presented to the programmer.

This is what the platform provides that the HAL does not. The boundary can sometimes be fuzzy, but the distinction is useful: HAL answers what the hardware can do, while the platform answers what the programmer wants to do.

SDKs as Domain-Specific Platform Abstractions

If the platform answers the general wishes of the programmer, an SDK answers the wishes of programmers working in a specific domain.

SDK stands for Software Development Kit. It is a platform abstraction focused on a particular problem domain. It provides tools, libraries and APIs that solve typical problems in that domain.

In digital television, this becomes a DTV SDK. In autonomous driving, a similar idea may appear as a DRIVE SDK. Different industries have different SDKs, and these SDKs often come from SoC makers because they must work with specific chips.

A well-known digital TV example is Broadcom's DTV SDK called Nexus. Such SDKs expose domain-specific abstractions for building TV functionality on top of the platform and hardware.

At the HAL level, the question was what the hardware can do. At the DTV SDK level, the question becomes: what does a programmer working in digital television want to do?

DTV SDK and Signal Path Abstractions

Project 1 introduced the idea of a dataflow diagram. This can also be called a signal path or a pipeline.

A pipeline is itself an abstraction. Building a signal path is an abstract operation. A good DTV SDK allows the programmer to construct such a signal path by combining processing blocks, connecting their inputs and outputs, and configuring the complete chain.

For example, a live TV playback configuration may contain a tuner, demux, optional descrambler, audio decoder, video decoder and output layers. Without a domain-specific SDK, the programmer would have to manually track every detail: which PID is used, how buffers are routed, how data is connected and which block receives which stream.

A DTV SDK can provide higher-level functions for building this configuration. The programmer may open the demux in a selected mode, connect the tuner to its input, connect selected outputs to the video and audio decoders, and create the required playback pipeline.

In such an SDK, processing blocks become software objects. The SDK allows them to be instantiated, connected and configured.

Instantiation is a platform concept, not a hardware concept. A system may expose two logical instances of a demux even if the underlying hardware contains one physical demux block. The platform or SDK maps logical instances to available hardware resources.

This also makes resource management a responsibility of the DTV SDK. When software calls open, the SDK must decide whether the requested object can be created. Opening

and closing are platform abstractions. Hardware itself does not have the concept of being opened. The platform creates a context, handle or identifier that represents the logical use of that resource.

Once the handle exists, further calls can use it. Multiple logical contexts may exist independently, even if they share hardware resources underneath.

This type of abstraction is exactly what DTV SDKs provide for blocks such as tuners, demuxes, audio decoders and video decoders.

Other SDKs and frameworks also appear in this space. Apart from Broadcom Nexus, platforms from vendors such as MStar, HiSilicon and others may include similar components. RDK is another important framework in the set-top box and digital TV domain.

Combining Platform Components in Real Systems

In real development, software stacks usually combine components from several layers and vendors.

For example, a Linux-based DTV product may use Linux as the operating-system base, WebKit as the application environment, SSL and network libraries for communication, and a DTV SDK such as Nexus for digital TV functionality. A WebKit plugin may then expose Nexus functionality to the web application. This allows a DTV application to combine web-based UI with native digital TV playback functionality.

This integration picture is common in media devices.

A/V SDKs and Playback Frameworks

A separate group of SDKs focuses explicitly on audio and video processing. Sometimes these are part of the DTV SDK, and sometimes they are separate.

These A/V SDKs provide abstractions for playback from files, IP streams and other sources. They are especially important because modern media products mix broadcast and broadband use cases.

On Android, this course will use MediaCodec and AudioTrack as media playback abstractions. On Linux, similar roles are often played by FFmpeg or GStreamer. These are platform-level abstractions for building playback pipelines.

The basic job of these frameworks is to feed data toward audio and video decoders. The data may come from a file, network stream, memory buffer or another source. The framework organizes the flow and feeds the decoders correctly.

One difficult area appears when these A/V SDKs must coexist with broadcast-style pass-through modes. Many A/V playback frameworks originally come from the Internet and file playback world, not from classical broadcast television. Their assumptions are shaped by

files, IP streams, adaptive streaming and cloud-style media delivery. Broadcast playback, however, involves tuners, demuxes and transport-stream hardware.

Because of this, there is often a mismatch between the DTV SDK world and the A/V playback framework world.

If the use case is broadcast playback in memory mode, the system may need to combine the DTV SDK and the A/V SDK. If the use case is pure IP playback or adaptive streaming, it may be possible to use MediaCodec directly without using the DTV SDK.

Conditional Access SDKs

Conditional access may have its own SDK. This is because key delivery, entitlement checking, authorization and descrambling are often provided by a separate vendor.

In real systems, the conditional access provider may deliver a dedicated SDK that must be integrated into the platform. These components are usually protected and distributed separately, because the security model depends on keeping internal mechanisms confidential.

For this lecture, the DTV SDK remains the central domain-specific platform layer, but A/V SDKs, conditional access SDKs and other specialized SDKs are also important parts of real media software stacks.

Middleware and Application Layer

The next large layer is middleware. It is called middleware because it sits between applications and the platform. It can also be understood as an application layer because it solves application-level problems.

Middleware deals with problems directly related to what the user wants to do. Some of these problems may be implemented inside the application itself, but many are implemented in frameworks that belong to middleware.

Application Runtime

The first important middleware component is the application runtime.

At the operating-system level, applications may be represented as processes and threads. Under Linux, an application often means a process with a graphical interface. However, an application runtime adds another abstraction level above that.

The runtime executes the application inside an environment that does not directly correspond to a raw operating-system process from the developer's point of view. Often, the application is written as a script or interpreted description of behavior, while the runtime executes it as part of a larger environment.

A practical test is whether the whole device must be reset if the application crashes. If it does not, the application is probably running inside some runtime environment that isolates it from the rest of the system.

Examples include Java virtual machines, browsers and other runtime environments. Android has its own runtime. WebOS is a Linux-based realization of this idea. Proprietary web-based runtimes also exist, such as iWedia Teatro.

DTV Middleware

Parallel to the application runtime, usually implemented as browser plugins, Android services or native services, there is DTV middleware.

Earlier lectures introduced metadata parsing, command and control and other abstract CPU tasks. Much of that functionality belongs to DTV middleware.

Channel scan is a typical middleware function. It is not solved by one simple platform function call. The platform may help create pipelines, open hardware resources and connect blocks, but the middleware manages the actual scanning logic, service database, frequencies, service parameters, currently active channel and channel switching state.

DTV middleware solves problems related to the execution state of the device. It tracks which service is currently active, which hardware blocks are used, which signal path is configured and what should happen when the user presses a button on the remote controller.

These state transitions and use-case-level configurations are middleware responsibilities.

On Android, an important framework in this area is the TV Input Framework. This course will work with it directly. iWedia also provides its own middleware implementation called Comedia. Other systems may include custom proprietary middleware.

OTT and IPTV systems also have client-side middleware. Whenever a client communicates with a server, obtains metadata, receives data and controls playback, some client-side software is responsible. Grouped together, this becomes an OTT or IPTV client.

DTV middleware can therefore appear as broadcast middleware, TV input framework, proprietary middleware, OTT client, IPTV client or similar software.

Media Players and UI Frameworks

Media players sit near the middleware layer and use lower playback frameworks and middleware services. Operating systems often provide their own media player abstractions. Android's MediaPlayer is one example.

In practice, a player is often instantiated in full screen, while user applications control it through high-level commands. Applications should not normally implement a complete

player from scratch. Instead, the player accepts commands such as play, stop, pause, resume, set URL or set the path to an MPD file.

UI frameworks also belong to the broad middleware and application-support world. In web environments, the developer may use HTML, CSS and JavaScript directly or use frameworks such as React. In Android, the developer uses Android UI frameworks. Other frameworks such as Flutter, Unity, Qt, GTK or wxWidgets may also appear depending on the platform and product.

These frameworks help create user interfaces, but they remain close to the application layer.

Middleware APIs: What Does the User Want?

At the HAL level, the question was: what can the hardware do?

At the platform level, the question was: what does the programmer want?

At the middleware level, the question becomes: what does the application, or ultimately the user, want?

The user wants to change from channel one to channel two. Middleware should make that possible. Ideally, the middleware exposes a high-level function such as `changeChannel`, where the application provides the channel number. Other functions may include `startScan`, `getChannelName` or `startPlayback`.

This is the middleware abstraction level.

DTV middleware also implements concrete standards such as DVB-T or ATSC. It parses metadata tables. For example, a PAT parser belongs in middleware, not down in the HAL layer. Metadata parsing, metadata management and state handling are middleware concerns.

Applications

At the top of the host software stack are applications.

A Live TV application will almost always exist on a TV or set-top box, and such an application will be used in this course. There may also be proprietary video-on-demand applications, operator applications, OTT applications such as Netflix or HBO Max, and media-player applications such as ExoPlayer.

The application should not perform heavy lifting related to standards, hardware control or low-level playback abstractions. Its primary role is to communicate with the user.

Conceptually, the application handles user events, triggers processes by calling middleware APIs, listens for events returned from middleware and updates the

presentation. It changes the graphical view, draws menus, windows and controls, and displays data produced by middleware.

If an application contains much more than user communication and presentation logic, then either the application design is questionable or the platform has forced too much responsibility upward.

Call Stack Across the Layers

The host software stack can be understood through a call hierarchy.

At the middleware level, software answers what the user wants. For example, the user wants to start a TV service. Middleware may expose a function such as `startService()` or a service-control operation with the same meaning.

This function is implemented inside DTV middleware. However, the middleware does not do everything alone. To implement the user-level operation, it calls platform functions underneath.

For example, in a DTV SDK such as Broadcom Nexus, there may be a function like `Nexus_ParserBand_SetSettings`. Parser Band is a Nexus abstraction related to demultiplexing. It can parse selected PIDs, and those PIDs are configured through settings passed to the function.

So the application may request a high-level operation such as starting a service. Middleware translates that into platform-level operations, such as configuring a parser band with selected PIDs. The DTV SDK then communicates with the actual demux driver through lower-level driver calls.

This should not be understood as a strict one-to-one mapping. One middleware call does not always correspond to exactly one SDK call, and one SDK call does not always correspond to exactly one driver call. The example only illustrates the hierarchy.

Higher layers usually have fewer, more meaningful calls. Lower layers execute longer and more concrete sequences of operations. At the driver level, this may involve many register writes, reads, waits, interrupts and low-level interactions.

The higher we go, the more abstract the operation becomes. The lower we go, the more hardware-specific it becomes.

Ownership of the Stack

Different organizations are responsible for different parts of the stack.

The operating system layer is often taken from open-source projects or provided by an operating-system vendor, especially if the OS is proprietary.

Much of the platform layer is usually delivered by the silicon company, meaning the SoC manufacturer. As part of the BSP, the board support package, the manufacturer provides not only hardware but also a large portion of platform software. In many modern projects, the silicon vendor provides the operating system together with the platform, already integrated and ready to use.

Conditional access is usually provided by a dedicated conditional access vendor, often in cooperation with the silicon company. Because these systems depend on security and secrecy, libraries are distributed separately and internal mechanisms are not fully visible.

Middleware is more mixed. Operating-system vendors may provide middleware, but it is often extended or replaced by solutions from specialized middleware companies, such as iWedia. These companies may also need to adjust the platform slightly because platforms are not perfectly abstracted in practice.

Ideally, the separation would be clean: the platform would be fixed and middleware would be portable. In practice, some platform adaptation is often needed.

Portability and Integration

Portability is one of the most important concepts in the software stack.

The platform itself is not truly portable because it is tied to hardware. Even if an API looks the same across different systems, the implementation depends on a specific SoC. It is not easy to take a platform implementation and move it to another hardware target.

Middleware, in contrast, is designed to be portable. The idea is that middleware should move from one platform to another with relatively low effort, as long as compatible APIs are available.

Applications are also expected to be portable within a given ecosystem.

At the top, applications are usually delivered by service providers, such as telecom operators, content providers or streaming companies. These organizations define and own the user experience, even if external engineering teams implement parts of the software.

Finally, there are integrators. Integration is often the most difficult and least glamorous part of the job. Integrators take hardware, platform, middleware and applications and make them work together as a complete system.

In real projects, this means dealing with mismatched APIs, incomplete implementations, missing features and various incompatibilities. This is where glue code appears: adaptation layers that connect components which were not originally designed to work perfectly together.

If the stack were ideal, integration would be easy. In reality, APIs between layers often do not align neatly, so integrators must bridge the gaps. This is why integration requires a deep understanding of the entire stack, from hardware to applications.

Project 2 Preview

The concepts introduced in this lecture directly prepare the second project.

For each assigned use case, students will first design a high-level software stack block diagram. The diagram should be as detailed as possible based on the current understanding of the stack.

The next task is to define example call stacks. This means describing how API calls propagate through the layers: from application, through middleware, through the platform, down to the hardware abstraction layer. These call stacks should be illustrated using UML sequence diagrams.

In the following phase of Project 2, the architecture designed on paper will be brought to life in software.

The idea is the same as in Project 1, but now the focus shifts from conceptual signal paths to concrete code, concrete function calls and real implementations. Students must understand which functions are called, how they are connected and how they are used correctly inside the stack.

Closing Perspective

This lecture introduced the host software stack of a digital TV and multimedia device. The CPU-side software was decomposed into drivers and HAL, platform and operating-system abstractions, common libraries, domain-specific SDKs, middleware, application runtimes, media players, UI frameworks and applications.

The key conceptual distinction is the question each layer answers. HAL answers what the hardware can do. The platform answers what the programmer wants. Middleware answers what the application or user wants. Applications present the final user experience.

Understanding this layered model is essential for moving from architecture diagrams to real media software implementation.